

Configuration et état complet du Chatbot (n8n) — CODRescue

Dernière mise à jour : 9 décembre 2025

Ce document rassemble l'ensemble des éléments nécessaires pour rendre le workflow n8n « Chatbot Webhook SQL query - Compatible Version » robuste et reproductible. Il contient :

- le diagnostic du problème rencontré
 - le prompt système et utilisateur recommandés
 - le JSON Schema à exiger pour les sorties de l'agent
 - des exemples (few-shot)
 - paramètres recommandés du modèle
 - mapping **Respond to Webhook**
 - checklist de tests et debugging
 - bonnes pratiques mémoire / sessions
-

1) Résumé du problème

Symptômes observés :

- n8n renvoyait l'erreur « Could not parse LLM output » ou « réponse non JSON ou vide ». Le modèle renvoyait du texte libre (ex : "Je suis prêt..."), et non un objet JSON structuré attendu par l'agent/les outils.
- Les réponses côté frontend pouvaient paraître « illogiques » ou incohérentes lorsque l'agent n'était pas contraint.

Cause principale : absence d'un prompt système précis et du format de sortie forcé (Require Specific Output Format). Le modèle produisait du texte naturel au lieu d'une structure JSON utilisable par n8n/LangChain.

2) Objectifs de la configuration

- Forcer des réponses structurées (JSON) pour permettre : exécution automatique de requêtes SQL, parsing fiable par n8n, et affichage stable côté frontend.
 - Eviter les hallucinations et sorties non déterministes.
 - Préserver la sécurité (pas d'UPDATE/DELETE sans validation).
-

3) Prompt Système recommandé (à coller dans **AI Agent** / System Prompt)

Copiez-coller intégralement le texte suivant dans le champ System Prompt du nœud **AI Agent** (ou SQL Agent) :

Vous êtes CODRescue Assistant, un assistant SQL strictement spécialisé dans l'analyse de données e-commerce pour CODRescue.

Contrainte principale : Répondez en français, soyez précis, et RESPECTEZ STRICTEMENT LE FORMAT DE SORTIE JSON DEMANDÉ CI-DESSOUS. NE PRODUISEZ AUCUN TEXTE HORS DU JSON.

Comportement attendu :

- Quand l'utilisateur pose une question, décidez si une requête SQL est nécessaire.
- Si oui, générez la requête SQL exacte dans la clé "sql".
- Exécutez la requête via les tools connectés (ou indiquez la requête dans "sql" si la partie exécution est séparée).
- Retournez les résultats dans "rows" (tableau d'objets) et un résumé en "answer" (phrase courte en français).

Règles additionnelles :

- Ne fournissez jamais d'informations inventées. Si vous n'avez pas accès à une donnée, mettez "rows" à null et expliquez dans "answer" pourquoi.
- Si une clarification est requise, renvoyez exactement :
{
 "answer": "clarify",
 "sql": null,
 "rows": null,
 "question": "<question de clarification>".
}
- Limitez "rows" à 50 lignes maximum (utilisez LIMIT dans le SQL).
- N'exécutez jamais de DELETE/UPDATE/ALTER/DROP sans validation explicite.
- Si vous ne comprenez pas la question, renvoyez le JSON de clarification ci-dessus.

Contexte minimal de la base : tables importantes (exemples) :

- `article\_article` (colonnes typiques : id, nom, reference, prix_unitaire, categorie_id)
- `article\_variantearticle` (article_id, couleur_id, pointure_id, qte_disponible)
- `commande\_commande` (num_cmd, date_cmd, total_cmd, client_id, etat_id)
- `commande\_panier` (commande_id, article_id, quantite, prix_panier)
- `client\_client` (id, nom, prenom, email, numero_tel)

Si besoin, utilisez l'outil "Table Schema" pour lire la structure d'une table avant de générer la requête.

Fin des instructions système.

4) Prompt utilisateur recommandé (champ **Prompt** / User Message)

Le prompt utilisateur peut rester court (il sera complété par le system prompt). Exemple :

Vous êtes un assistant SQL spécialisé dans l'analyse de données e-commerce pour CODRescue. Répondez en français et soyez précis.

Mais il est préférable de fournir quelques détails additionnels ou un contexte (ex : période demandée) depuis le frontend dans `chatInput`.

5) JSON Schema (Require Specific Output Format)

Collez ce JSON Schema dans l'option "Require Specific Output Format" du nœud **AI Agent** :

```
{
  "type": "object",
  "properties": {
    "answer": { "type": "string", "minLength": 1 },
    "sql": { "anyOf": [{ "type": "string" }, { "type": "null" }] },
    "rows": { "anyOf": [{ "type": "array", "items": { "type": "object" } }, {
"type": "null" }] },
    "question": { "anyOf": [{ "type": "string" }, { "type": "null" }] }
  },
  "required": ["answer", "sql", "rows"],
  "additionalProperties": false
}
```

Ce schéma force un objet JSON exact contenant `answer`, `sql` et `rows` (ou `question` pour clarifications).
n8n / LangChain valideront le format et échoueront si le modèle produit du texte libre.

6) Few-shot : exemples à inclure dans le prompt (apprendre au modèle)

Ajoutez ces 3 exemples courts après le prompt système pour guider l'agent :

Exemple 1 - Question simple:

User: "Combien d'articles existe-t-il dans le catalogue ?"

Réponse attendue:

```
{"answer":"Le nombre total d'articles est 1234.","sql":"SELECT COUNT(*) AS cnt
FROM article_article;","rows":[{"cnt":1234}]}
```

Exemple 2 - Analyse précise:

User: "Quels sont les 3 articles les plus vendus ce mois ?"

Réponse attendue:

```
{"answer":"Les 3 articles les plus vendus ce mois sont ...","sql":"SELECT
article_id, SUM(quantite) AS total FROM commande_panier WHERE date_cmd >=
date_trunc('month', current_date) GROUP BY article_id ORDER BY total DESC LIMIT
3;","rows":[{"article_id":123,"total":50},{"article_id":45,"total":31},
{"article_id":78,"total":20}]}
```

Exemple 3 - Clarification:

User: "Montre-moi les ventes."

Réponse attendue:

```
{"answer": "clarify", "sql": null, "rows": null, "question": "Veuillez préciser la période (ex: aujourd'hui, ce mois, année)."}"
```

7) Paramètres modèle recommandés

- Temperature : 0.0 → 0.2 (0.0 pour production, comportement déterministe)
- Max tokens / length : 1024 (ou selon modèle disponible)
- Top_p : 0.8 (optionnel)
- Max iterations (si agent plan/execute) : 5–10
- Timeout : 30–60s (si les requêtes SQL peuvent être longues)

Réduire la température réduit le risque de réponses non structurées.

8) Mapping **Respond to Webhook** (exemples d'expressions)

Dans le nœud **Respond to Webhook** :

- **Respond With** → **JSON**
- **Response Body** (mode expression) :

Si l'objet final est exposé par le nœud **AI Agent** dans la propriété **output** :

```
{{ $node["AI Agent"].json["output"] }}
```

Si l'AI retourne du texte dans une structure **response.generations...text** (format modèle), et que vous voulez renvoyer ce texte brut temporairement :

```
{{ { "output": $node["AI Agent"].json["response"]["generations"]  
[0].generations[0].text } }}
```

Remarque : préférez renvoyer l'objet JSON validé par le schéma. Le fallback texte est uniquement pour debug rapide.

9) Mémoire / Session

- Assurez-vous que le nœud Memory (ex : **Simple Memory** ou **Postgres Chat Memory**) possède une Session ID valide. Exemple d'expression :

```
{{ $json.body.sessionId }}
```

- Conservez une fenêtre contextuelle courte (context window = 10 derniers messages) pour éviter que l'agent ne soit surchargé.

10) Tests et procédure de debugging (pas à pas)

1. Exécutez localement le nœud **AI Agent** (bouton **Execute Step**) et inspectez **Output** et **intermediateSteps**.
2. Vérifiez que la propriété exposée (**output**) contient un JSON valide conforme au schema.
3. Si la sortie n'est pas JSON :
 - baissez **temperature** à 0.0
 - ajoutez en tête du prompt : "REPLY WITH ONLY THE JSON OBJECT AND NOTHING ELSE." (en majuscules si besoin)
 - réexécutez.
4. Si le modèle répond par une question de clarification (format **answer: "clarify"**), redirigez la question vers le frontend puis renvoyez la nouvelle requête.
5. Test d'intégration : envoyez une requête depuis le frontend (ex : powershell ou via l'UI) et vérifiez affichage correct dans le chat.

Exemple PowerShell pour tester webhook :

```
$body = @{ chatInput = 'je veux le nombre total des articles'; sessionId = 'test-session-001' } | ConvertTo-Json

Invoke-WebRequest -Uri 'https://YOUR_WEBHOOK_URL' -Method POST -Body $body -Content-Type 'application/json'
```

Vérifiez la réponse retournée par n8n : JSON conforme, puis le frontend affichera le champ **answer** ou formatera **rows**.

11) Checklist de production

- ☐ System Prompt collé
- ☐ JSON Schema activé
- ☐ Few-shot ajouté
- ☐ Temperature réglée à 0.0–0.2
- ☐ Memory Session configurée
- ☐ Tools SQL (Execute Query, Table Schema, Table List) connectés & sélectionnés
- ☐ Respond to Webhook mappe l'objet JSON valide
- ☐ Tests unitaires et d'intégration effectués

12) Bonnes pratiques et recommandations

- Fournir les schémas de tables essentielles au modèle (ou permettre au modèle d'appeler le tool Table Schema) pour réduire les erreurs.

- Ne pas exposer les credentials en clair dans les prompts ou logs.
 - Journaliser (**logging**) chaque **sql** exécutée et son résultat pour audit.
 - Pour des requêtes lourdes, considérer la mise en cache des réponses fréquemment demandées.
-

13) Prochaines étapes proposées

1. Coller le System Prompt + JSON Schema dans le nœud **AI Agent**.
2. Exécuter **AI Agent** isolément et vérifier **output** valide.
3. Mettre à jour **Respond to Webhook** pour renvoyer l'objet validé.
4. Effectuer tests depuis le frontend et ajuster few-shot au besoin.

Si vous le souhaitez, je peux :

- générer une version plus concise du prompt pour l'équipe technique ;
 - créer un petit script de test automatique (curl/powershell) qui exécute des scénarios et valide les réponses JSON ;
 - ajouter un bloc **Table Schema** à exposer automatiquement au modèle (si vous me fournissez les colonnes critiques, je les intègre dans le prompt).
-

Contacts / Ressources

- Documentation LangChain troubleshooting:
https://js.langchain.com/docs/troubleshooting/errors/OUTPUT_PARSING_FAILURE/
 - n8n Docs: <https://docs.n8n.io/>
-

Fichier généré par l'assistant : **docs/n8n_agent_full_state.md** — collez les blocs dans vos nœuds n8n et testez. Si vous souhaitez que j'applique ces modifications directement dans un fichier existant du repo, dites-le et je peux l'ajouter/modifier via patch.